

Vyomi Provider Gap Matrix

This matrix is the current parity backlog for Vyomi. It is aligned to official provider documentation so implementation work can be checked against the vendor API, utility, SDK, and console references.

Official Reference Set

AWS

- Console: [AWS Management Console](#)
- CLI: [AWS CLI documentation](#)
- Java SDK: [AWS SDK for Java 2.x](#)
- Go SDK: [AWS SDK for Go v2](#)
- Core service APIs:
 - [EC2 API Reference](#)
 - [S3 API Reference](#)
 - [DynamoDB API Reference](#)
 - [API Gateway docs](#)
 - [AWS SDK for Java 2.x docs](#)
 - [AWS SDK for Go v2 docs](#)

GCP

- Console: [Google Cloud console](#)
- CLI: [gcloud reference](#)
- Legacy storage utility: [gsutil](#)
- Client libraries: [Cloud Client Libraries](#)
- Core service APIs:
 - [Compute Engine API usage](#)
 - [Cloud Storage APIs](#)
 - [Cloud SQL Admin API](#)
 - [Pub/Sub service APIs overview](#)
 - [Firestore APIs](#)
 - [Cloud Functions REST reference](#)
 - [API Gateway overview](#)
 - [IAM APIs](#)

API Parity

AWS

Surface	Official reference	Current state	Next step
EC2	EC2 API Reference	Integrated	Verify request/response shapes for instance lifecycle, SSH/console access, and Query aliases against the official API examples.
S3	S3 API Reference	Integrated	Validate bucket/object/list/versioning/ACL-style behavior against the S3 REST contract.
IAM	IAM docs	Integrated	Expand policy/attachment/account settings parity to match the service documentation for all supported resource types.
VPC	Amazon VPC docs	Integrated	Confirm VPC, subnet, route table, security group, and internet gateway semantics against the official API model.
RDS	Amazon RDS docs	Integrated	Reconcile database, snapshot, subnet group, and parameter group workflows with the AWS API examples.
Lambda	AWS Lambda docs	Integrated	Validate function versioning, invoke, permissions, and update flows against the official Lambda API behavior.
SQS	Amazon SQS docs	Integrated	Check queue, message, purge, and tagging semantics against the queue API reference.
DynamoDB	DynamoDB API Reference	Integrated	Verify JSON item shapes, query/scan behavior, and table state transitions against the official API reference.
API Gateway	API Gateway docs	Integrated	Validate resources, methods, deployments, stages, and invoke paths against the official REST API model.

GCP

Surface	Official reference	Current state	Next step
Compute Engine	Compute Engine API usage	Integrated	Validate instance lifecycle, long-running operations, start/stop/reset/delete, and zone/project semantics against the official API docs.
Cloud Storage	Cloud Storage APIs	Integrated	Verify bucket/object CRUD, media upload, and metadata handling against the Cloud Storage JSON API.
Cloud SQL	Cloud SQL Admin API	Integrated	Check instance lifecycle and restart/delete semantics against the Admin API docs.
Pub/Sub	Pub/Sub service APIs overview	Integrated	Validate topics/subscriptions/publish/pull/ack/modifyAckDeadline against the documented service APIs.

Surface	Official reference	Current state	Next step
Firestore	Firestore APIs	Integrated	Reconcile document and query behavior with the native Firestore API model.
Cloud Functions	Cloud Functions REST reference	Integrated	Confirm function create/call/delete/IAM/policy operations against the REST reference.
API Gateway	API Gateway overview	Integrated	Validate API/config/gateway lifecycle flows against the documented API Gateway model.
VPC Network	Compute Engine API usage	Integrated	Check network, subnetwork, and firewall behavior against the Compute Engine networking APIs.
IAM	IAM APIs	Integrated	Verify policy and service-account behavior against the IAM API docs.

Utilities Parity

AWS

Utility	Official reference	Current state	Next step
AWS CLI	AWS CLI documentation	Integrated	Expand coverage of command forms, output shapes, and error behavior to match the CLI command reference for the supported services.
SDK bootstrap snippets	AWS SDK for Java 2.x / AWS SDK for Go v2	Partial	Keep the bootstrap examples, then add true transport adapters so Java/Go clients can talk to the simulator without request rewriting.
Boto3 / botocore compatibility	AWS CLI + service API refs	Partial	Add request-shape adapters for the AWS API surfaces that are already implemented.

GCP

Utility	Official reference	Current state	Next step
gcloud	gcloud reference	Partial	Expand command-form parity for compute, storage, SQL, Pub/Sub, Firestore, Functions, API Gateway, VPC, and IAM workflows.

Utility	Official reference	Current state	Next step
gsutil	gsutil docs	Planned / legacy	Support legacy storage workflows only where needed; prefer <code>gcloud storage</code> for new parity work because Google documents it as the primary modern path.
Cloud Client Libraries bootstrap snippets	Cloud Client Libraries	Partial	Keep the Java/Go snippets, then add transport adapters and authentication defaults that match the client library guidance.
<code>gcloud storage</code> alignment	Cloud Storage APIs	Partial	Make the storage flows align with the modern <code>gcloud storage</code> and Cloud Storage API contract before treating <code>gsutil</code> as a first-class target.

SDK Parity

Provider	SDK	Official reference	Current state	Next step
AWS	Java	AWS SDK for Java 2.x	Partial	Add transport adapters and request builders for the supported service clients so the SDK samples work with the simulator endpoint.
AWS	Go	AWS SDK for Go v2	Partial	Add transport adapters and region/endpoint handling so the SDK samples can use the simulator directly.
GCP	Java	Cloud Client Libraries	Partial	Add transport adapters for Compute, Storage, SQL, Pub/Sub, Firestore, Functions, IAM, and API Gateway workflows.
GCP	Go	Cloud Client Libraries	Partial	Add transport adapters and endpoint configuration for the supported GCP client libraries.

Console Parity

AWS

Area	Official reference	Current state	Next step
Console home / navigation	AWS Management Console	Partial	Keep aligning the console shell, unified navigation, service search, and widget-style summaries with AWS Console Home.

Area	Official reference	Current state	Next step
Service consoles	AWS Management Console	Partial	Tighten the remaining service pages that still use simplified layouts; IAM, S3, VPC, and the AWS create flows for RDS, SQS, DynamoDB, API Gateway, and Lambda now have console-style framing, breadcrumbs, and summary chips.
Resource graph	AWS Management Console	Partial	Keep the graph provider-separated, draggable, and readable; add only provider-native details that support console parity.

GCP

Area	Official reference	Current state	Next step
Console home / project dashboard	Google Cloud console	Partial	Keep matching project dashboard behavior, per-project chips, and the API dashboard model documented by Google.
Service-specific pages	Google Cloud console	Partial	Tighten the remaining simplified layouts so they follow Google Cloud's page structure and detail panes more closely; Compute Engine and Cloud SQL now use a stacked list-then-details pattern, while Functions, API Gateway, Pub/Sub, and Firestore keep their richer action rails and summary chips.
Resource graph	Google Cloud console	Partial	Keep the graph provider-separated, topology-based, and readable as the resource count grows.

CloudSim Backbone

Surface	Status	Notes
Spaces / federation	Integrated	Space isolation, linking, budgets, reconcile, and active-space EC2 launch policy gates are present.
Provider registry / surface registry	Integrated	AWS, Azure, GCP, and Other are separated in metadata and surfaced through the provider matrix API.
Provider packs	Integrated	AWS and GCP packs are provider-namespaced; Azure/Other remain reserved surfaces.
Provider helpers	Integrated	<code>providers/aws_routes.py</code> , <code>providers/gcp_routes.py</code> , <code>providers/aws_ec2_routes.py</code> , <code>providers/gcp_compute_routes.py</code> , <code>providers/aws_iam.py</code> , <code>providers/gcp_iam.py</code> , <code>providers/aws_vpc.py</code> ,

Surface	Status	Notes
		<code>providers/aws_rds.py</code> , <code>providers/gcp_storage_sql_vpc.py</code> , <code>providers/aws_services.py</code> , <code>providers/gcp_services.py</code> , <code>providers/capabilities.py</code> , and <code>core/tooling_simulators.py</code> expose the modular provider/tooling entry points.
Provider capabilities	Integrated	<code>/api/providers/{provider}/services</code> and <code>/api/providers/{provider}/capabilities</code> surface the AWS and GCP service catalogs with route maps.
Provider routers	Partial	Route registration is modular, and the SQS queue, DynamoDB, Lambda invoke chain, and API Gateway invoke path now live in <code>providers/aws_services.py</code> , but Lambda management helpers and API Gateway snapshot/deployment helpers still live in <code>server.py</code> .

Execution Checklist

API Parity

- ☒ Build service-by-service API contract checks from the official AWS and GCP references above. See [tests/test_api_parity_contracts.py](#).
- ☐ Validate AWS EC2, S3, IAM, VPC, RDS, Lambda, SQS, DynamoDB, and API Gateway against the official request/response shapes.
- ☐ Validate the CloudSim active-space policy gate for EC2 launches so denied spaces never start a host runtime.
- ☐ Validate GCP Compute Engine, Cloud Storage, Cloud SQL, Pub/Sub, Firestore, Cloud Functions, API Gateway, VPC Network, and IAM against the official REST/API docs.
- ☐ Add explicit unsupported-operation markers wherever the official docs describe behavior we do not yet simulate.

Utilities Parity

- ☐ Expand the AWS CLI translator to match the documented command groups and output shapes for the supported services.
- ☐ Expand `gcloud` command-form coverage for the supported GCP services.
- ☐ Keep `gsutil` as legacy storage compatibility only, and prefer `gcloud storage` for new parity work.
- ☐ Convert the snippet endpoints into true transport adapters for SDK-driven workflows.

SDK Parity

- ☐ Add Boto3/botocore request-shape adapters for AWS parity.
- ☐ Add strict AWS Java and Go SDK transport adapters.
- ☐ Add strict GCP Java and Go Cloud Client Library transport adapters.
- ☐ Make the client examples in the UI and docs point at the same simulator endpoint and auth model.

Console Parity

- ☐ Keep tightening AWS and GCP service pages using the official console docs as the layout baseline. AWS IAM, S3, VPC, and the AWS create flows for RDS, SQS, DynamoDB, API Gateway, and Lambda now use console framing; GCP Functions, API Gateway, Pub/Sub, and Firestore now include stronger action rails and project/location summaries.
- ☐ Preserve provider-separated topology graphs and detail tooltips.
- ☐ Keep the action rails and detail stacks only where they match the documented console workflow.

Backend Split

- ☐ Move the remaining Lambda management helpers and API Gateway snapshot/deployment helpers out of `server.py` into provider modules.
- ☐ Keep the route layer modular and make `server.py` a bootstrap and shared-state endpoint only.

Terraform Bridge

- ☒ Add a draft Terraform export endpoint for the active space. See `/api/terraform/export` and `core/terraform_export.py`.
- ☒ Expand the Terraform exporter so AWS, GCP, and Azure all emit provider-valid resource blocks for the supported resource graph.
- ☒ Add a Terraform workflow modal with download, plan, and apply actions in the UI.
- ☒ Add Terraform plan/apply workflow support in the UI.
- ☒ Add Terraform import/round-trip support back into the simulator graph.